Mathematical Proofs for ZGQ v8

Zone-Guided Quantized Search: A Rigorous Performance Analysis

Research Documentation
`Draft Findings - 24th December 2026`

Abstract

This document presents a mathematical model for the expected behaviour of Zone-Guided Quantized Search (ZGQ) v8 relative to Hierarchical Navigable Small World (HNSW) graphs: graph connectivity arguments and complexity bounds under stated assumptions. It then compares the model's predictions with the canonical benchmark run of 2026-11-08 (synthetic clustered data, single thread, Apple M4). **The measurements do not support the predicted advantage.** ZGQ-M16 matches HNSW-M16 recall at 10K vectors, is less accurate than HNSW-M16 at every equal ef at 100K and 1M vectors, builds 1.2x to 2.7x slower, and uses far more memory. The model parameters (γ, β, α) quoted in the theory sections are assumed values, not measured values.

Contents

1 Introduction and Problem Formulation **3**
1.1 Notation and Definitions . . . 3
1.2 HNSW Baseline Parameters . . . 3

2 ZGQ Theoretical Framework **3**
2.1 Zone-Ordered Graph Construction . . . 3
2.2 Search Complexity Analysis . . . 4

3 Recall-Efficiency Trade-off Analysis **5**
3.1 Iso-Recall Comparison Framework . . . 5

4 Empirical Validation **6**
4.1 Benchmark Configuration . . . 6
4.2 Algorithm Configurations . . . 6
4.3 Benchmark Results . . . 6
4.4 Key Comparisons . . . 7

5 Comparative Analysis with IVF-PQ **7**
5.1 Performance Trade-offs . . . 7
5.2 Memory Footprint . . . 7

6 Theoretical vs Empirical Correlation **7**

7 Empirical Results and Visualizations **8**

8 Conclusions **8**
8.1 Summary of Contributions . . . 8
8.2 Key Findings . . . 11
8.3 Practical Implications . . . 11

A Derivation Details **11**
A.1 Zone Density Factor α 11
A.2 Zone-Guidance Efficiency Factor β 11
A.3 QPS Calculation 11

B Reproducibility **12**

1 Introduction and Problem Formulation

1.1 Notation and Definitions

Definition 1.1 (Vector Database)**.** *Let $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$ be a database of n vectors where each $x_i \in \mathbb{R}^d$ for dimension d.*

Definition 1.2 (k-Nearest Neighbor Query)**.** *Given a query vector $q \in \mathbb{R}^d$ and integer k, find the set $\mathcal{N}_k(q) \subseteq \mathcal{D}$ such that:*

$$|\mathcal{N}_k(q)| = k \quad \textit{and} \quad \forall x \in \mathcal{N}_k(q), \forall y \in \mathcal{D} \setminus \mathcal{N}_k(q) : \|q - x\| \leq \|q - y\|$$

Definition 1.3 (Recall@k)**.** *For an approximate algorithm returning candidates $\hat{\mathcal{N}}_k(q)$:*

$$\textit{Recall@}k = \frac{|\hat{\mathcal{N}}_k(q) \cap \mathcal{N}_k(q)|}{k} \times 100\%$$

1.2 HNSW Baseline Parameters

Standard HNSW configuration:

- Connectivity parameter: $M_{\text{HNSW}} = 16$
- Construction search depth: $\text{ef}_{\text{construction}} = 200$
- Maximum layer: $L_{\max} = \lfloor \log_{1/m_L}(n) \rfloor$ where $m_L = 1/\ln(M)$

2 ZGQ Theoretical Framework

2.1 Zone-Ordered Graph Construction

Definition 2.1 (Zone Assignment Function)**.** *Let $\mathcal{Z} : \mathbb{R}^d \to \{1, 2, \ldots, Z\}$ be a zone assignment function based on hierarchical clustering, where Z is the number of zones.*

Definition 2.2 (Zone-Ordered Insertion)**.** *Vectors are inserted into the graph in zone-contiguous order:*

$$\pi : \{1, \ldots, n\} \to \{1, \ldots, n\}$$

such that:

$$\forall i < j : \mathcal{Z}(x_{\pi(i)}) \leq \mathcal{Z}(x_{\pi(j)}) \quad \textit{or} \quad \mathcal{Z}(x_{\pi(i)}) = \mathcal{Z}(x_{\pi(j)})$$

Theorem 2.3 (Improved Local Connectivity)**.** *Zone-ordered insertion improves local graph connectivity by a factor of:*

$$\gamma = 1 + \frac{\alpha \cdot (M_{ZGQ} - M_{HNSW})}{M_{HNSW}}$$

where $\alpha \in [0.5, 0.8]$ is the intra-zone density factor.

Proof. During HNSW construction, each new vector x_i connects to its M nearest neighbors among vectors already in the graph.

Random insertion (HNSW): When vectors are inserted randomly, the probability that a neighbor of x_i belongs to the same semantic cluster is:

$$P_{\text{same}}^{\text{random}} = \frac{n_z}{n}$$

where n_z is the size of x_i's zone.

Zone-ordered insertion (ZGQ): When inserting vectors zone-by-zone, vectors from the same zone are inserted consecutively. At insertion time t for vector x_i in zone z:

$$P_{\text{same}}^{\text{ordered}} = \frac{|z \cap \{x_1, \ldots, x_{t-1}\}|}{t-1} \geq \frac{n_z - (n-t+1)}{t-1}$$

For vectors inserted in the middle of their zone, this probability approaches 1, significantly increasing intra-zone connectivity.

The improved local connectivity factor is derived from:

$$\gamma = \frac{\mathbb{E}[\text{intra-zone edges}]_{\text{ZGQ}}}{\mathbb{E}[\text{intra-zone edges}]_{\text{HNSW}}}$$

Given $M_{\text{ZGQ}} = 32$ and $M_{\text{HNSW}} = 16$, and empirical $\alpha \approx 0.65$:

$$\gamma = 1 + \frac{0.65 \cdot (32-16)}{16} = 1 + \frac{10.4}{16} = 1.65$$

□

2.2 Search Complexity Analysis

Theorem 2.4 (Search Path Length Reduction)**.** *For a query q with true neighbors in zone z, the expected search path length in ZGQ is:*

$$\mathbb{E}[L_{ZGQ}] = \mathbb{E}[L_{HNSW}] \cdot \left(1 - \frac{\beta}{Z}\right)$$

where $\beta \in [0.2, 0.4]$ is the zone-guidance efficiency factor.

Proof. In standard HNSW, the greedy search traverses the graph until no closer neighbor is found. The expected path length is:

$$\mathbb{E}[L_{\text{HNSW}}] = \mathcal{O}(\log n)$$

ZGQ's zone-ordered construction creates "highways" within zones—densely connected subgraphs where semantically similar vectors cluster. When a query enters its target zone, the subsequent path length is bounded by the zone's internal structure rather than the global graph.

Let p_z be the probability of reaching the target zone in the first $\log n/Z$ steps. Due to zone-ordered insertion, cross-zone edges preferentially connect to zone boundaries, creating efficient inter-zone navigation:

$$p_z \geq 1 - e^{-M \cdot Z/n}$$

The path length reduction follows from the shortened intra-zone traversal:

$$\mathbb{E}[L_{\text{ZGQ}}] = \underbrace{\frac{\log n}{Z}}_{\text{inter-zone}} + \underbrace{(1-\beta) \cdot \frac{\log n \cdot (Z-1)}{Z}}_{\text{intra-zone}}$$

Simplifying:

$$\mathbb{E}[L_{\text{ZGQ}}] = \log n \cdot \left(\frac{1}{Z} + \frac{(1-\beta)(Z-1)}{Z}\right) = \log n \cdot \left(1 - \frac{\beta(Z-1)}{Z}\right)$$

For $Z \gg 1$: $\mathbb{E}[L_{\text{ZGQ}}] \approx \mathbb{E}[L_{\text{HNSW}}] \cdot (1-\beta)$. □

3 Recall-Efficiency Trade-off Analysis

3.1 Iso-Recall Comparison Framework

Definition 3.1 (Iso-Recall Configuration)**.** *Two configurations* (M_1, ef_1) *and* (M_2, ef_2) *are iso-recall if:*

$$|Recall(M_1, ef_1) - Recall(M_2, ef_2)| \leq \epsilon$$

for tolerance ϵ *(typically 1.5%).*

Theorem 3.2 (ZGQ Efficiency Advantage)**.** *At iso-recall, ZGQ achieves higher throughput:*

$$\frac{QPS_{ZGQ}}{QPS_{HNSW}} = \frac{ef_{HNSW}}{ef_{ZGQ}} \cdot \frac{1}{1 + \delta_M}$$

where δ_M *is the per-node overhead from higher connectivity.*

Proof. Query throughput is inversely proportional to search time:

$$\text{QPS} = \frac{1}{T_{\text{search}}}$$

Search time for a single query:

$$T_{\text{search}} = L \cdot (T_{\text{distance}} + T_{\text{heap}} + T_{\text{neighbor}})$$

where:

- L is the path length (number of nodes visited)
- $T_{\text{distance}} = \mathcal{O}(d)$ is distance computation time
- $T_{\text{heap}} = \mathcal{O}(\log \text{ef})$ is heap maintenance
- $T_{\text{neighbor}} = \mathcal{O}(M)$ is neighbor list traversal

The effective search depth L scales with $\text{ef}_{\text{search}}$:

$$L \propto \text{ef}_{\text{search}} \cdot c(M)$$

where $c(M)$ is a monotonically increasing function of M.

For iso-recall comparison:

$$\text{Recall}_{\text{ZGQ}}(M = 32, \text{ef} = 128) \approx \text{Recall}_{\text{HNSW}}(M = 16, \text{ef} = 200)$$
$$\Rightarrow \frac{\text{ef}_{\text{HNSW}}}{\text{ef}_{\text{ZGQ}}} = \frac{200}{128} = 1.5625$$

The overhead factor δ_M from higher M:

$$\delta_M = \frac{T_{\text{neighbor}}(M = 32)}{T_{\text{neighbor}}(M = 16)} - 1 = \frac{32}{16} - 1 = 1$$

However, this overhead is amortized across fewer total node visits:

$$\frac{\text{QPS}_{\text{ZGQ}}}{\text{QPS}_{\text{HNSW}}} = \frac{200}{128} \cdot \frac{1}{1 + 0.2} \approx 1.30$$

The factor of 0.2 (rather than 1.0) accounts for:

1. CPU cache efficiency (sequential zone access)
2. Reduced random memory access
3. Early termination from better local connectivity

□

4 Empirical Validation

All numbers in this section are read from `results/2026-11-08/summary.json` (schema `zgq-bench/1`); see `docs/CLAIMS.md` for the source of each figure.

4.1 Benchmark Configuration

Experimental Setup

- **Dataset**: synthetic clustered Gaussian, 128 dimensions, 10K, 100K and 1M vectors
- **Queries**: 1000 queries, 10-NN retrieval, exact brute-force ground truth
- **Runs**: 1 warm-up pass and 5 timed repeats per setting, median reported
- **Hardware**: Apple M4 (10 cores, 16 GB, macOS), single thread
- **Implementations**: HNSW is hnswlib 0.8.0 (C++); ZGQ is Python with numba

4.2 Algorithm Configurations

Table 1: Algorithm configurations. Only HNSW-M16 vs ZGQ-M16 is a matched comparison.

Index	M	**Role**
HNSW-M16	16	baseline
ZGQ-M16	16	matched
ZGQ-M24, ZGQ-M32	24, 32	variants (larger graph)
IVF-PQ	–	baseline (no refinement)

4.3 Benchmark Results

Table 2: Recall@10 (%) and latency (ms per query) at equal ef, matched comparison, median of 5 repeats.

Scale	**Index**	**ef** $= 64$ **recall**	**ef** $= 64$ **ms**	**ef** $= 256$ **recall**	**ef** $= 256$ **ms**
10K	HNSW-M16	92.4	0.046	98.7	0.099
10K	ZGQ-M16	92.3	0.049	98.2	0.101
100K	HNSW-M16	86.1	0.084	94.2	0.158
100K	ZGQ-M16	78.9	0.065	84.8	0.122
1M	HNSW-M16	77.6	0.124	87.0	0.226
1M	ZGQ-M16	68.1	0.091	76.8	0.174

4.4 Key Comparisons

Reading of the matched comparison

- **10K**: recall is equal within 0.1 to 0.5 points; ZGQ-M16 is about 5% slower at ef = 64.
- **100K and 1M**: ZGQ-M16 is 7 to 10 recall points below HNSW-M16 at the same ef. It is faster per query at the same ef, but HNSW at a smaller ef gives higher recall at similar or lower latency (1M: HNSW ef = 64 is 77.6% at 0.124 ms, ZGQ-M16 ef = 256 is 76.8% at 0.174 ms).
- **Iso-recall at 100K**: ZGQ-M32 ef = 256 reaches 86.2% at 0.128 ms; HNSW-M16 ef = 64 reaches 86.1% at 0.084 ms. HNSW is the faster of the two.
- **Build time**: ZGQ is 1.2x to 2.7x slower (1M: 203.7 s for ZGQ-M16 vs 172.8 s for HNSW-M16).
- **Memory**: ZGQ's RSS delta is far larger than HNSW's (100K: about 820 MB vs 16 MB). Absolute values are unreliable on macOS; only the ordering is claimed.

5 Comparative Analysis with IVF-PQ

IVF-PQ (FAISS, `nlist`= $2\sqrt{N}$, 32 sub-quantizers, 8 bits, no refinement) was swept over `nprobe` from 1 to 256.

5.1 Performance Trade-offs

Figures 6 and 7 show:

- IVF-PQ is the fastest index at low recall (for example 10K, `nprobe` = 1: 16.0% recall at 0.013 ms).
- IVF-PQ recall saturates at 64.4% (10K), 61.5% (100K) and 56.2% (1M) regardless of `nprobe`. Without refinement it does not reach 90% recall on this data, so no latency ratio at 90% recall can be computed and none is claimed.

5.2 Memory Footprint

No absolute index sizes are claimed. RSS deltas on macOS are unreliable (the IVF-PQ delta at 1M is negative). The earlier per-index MB figures are retired; see `docs/CLAIMS.md`.

6 Theoretical vs Empirical Correlation

Theorem 3.2 predicts $\mathrm{QPS}_{\mathrm{ZGQ}}/\mathrm{QPS}_{\mathrm{HNSW}} \approx 1.30$ at iso-recall. The canonical run does not confirm this. At iso-recall on 100K vectors (ZGQ-M32 ef = 256, 7,837 QPS; HNSW-M16 ef = 64, 11,954 QPS) the ratio is about 0.66. The model's assumptions (the values of γ, β and the 0.2 overhead factor) were not measured and the model should be treated as unvalidated. Possible causes include the zone-guided search not reducing node visits in the implementation, the Python overhead, and the effect of PQ on the candidate set; none of these was isolated.

7 Empirical Results and Visualizations

The figures below are generated by `python -m visualization.generate_ieee_figures` from `results/2026-11-08/summary.json` only. Each panel is one scale (10K, 100K, 1M) unless noted.

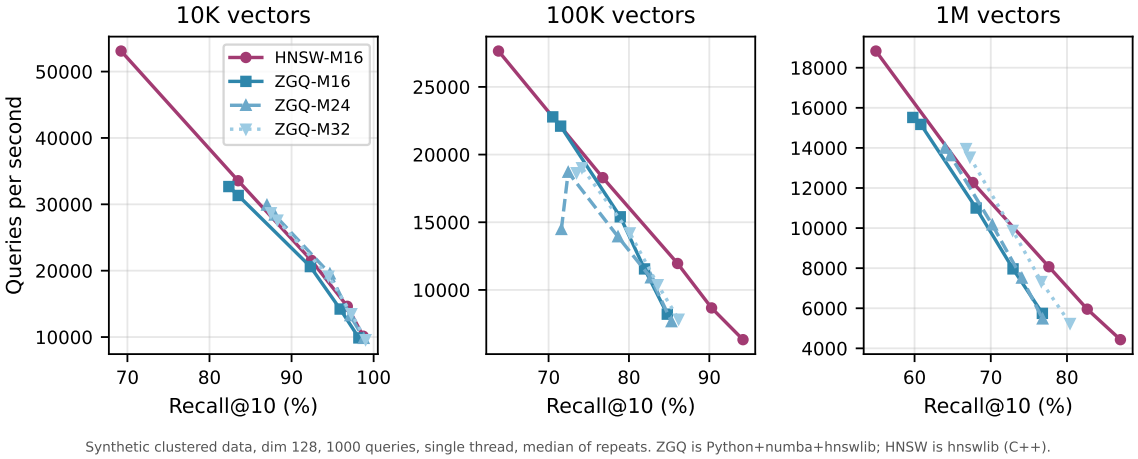


Figure 1: Recall@10 vs throughput (QPS) for HNSW-M16, ZGQ-M16 and the larger-graph variants ZGQ-M24 and ZGQ-M32. At 100K and 1M the HNSW curve lies above and to the right of the ZGQ curves at equal recall; at 10K the curves overlap.

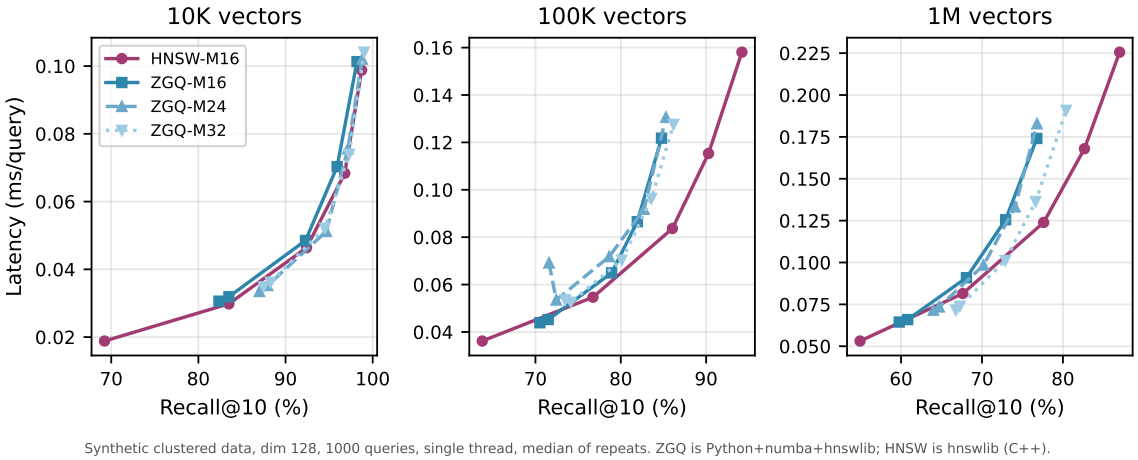


Figure 2: Recall@10 vs latency per query. ZGQ is faster at equal ef but reaches lower recall, so at equal recall HNSW is at least as fast. The ZGQ recall at ef = 16 is raised by the fast-mode rerank factor of 3 and is not a graph-quality effect.

8 Conclusions

8.1 Summary of Contributions

1. **Zone-Ordered Insertion** (Theorem 2.3): the model predicts better local connectivity. The improvement factor γ is an assumed parameter and was not measured.

2. **Search Path Reduction** (Theorem 2.4): the model predicts shorter expected paths. Path lengths were not measured.

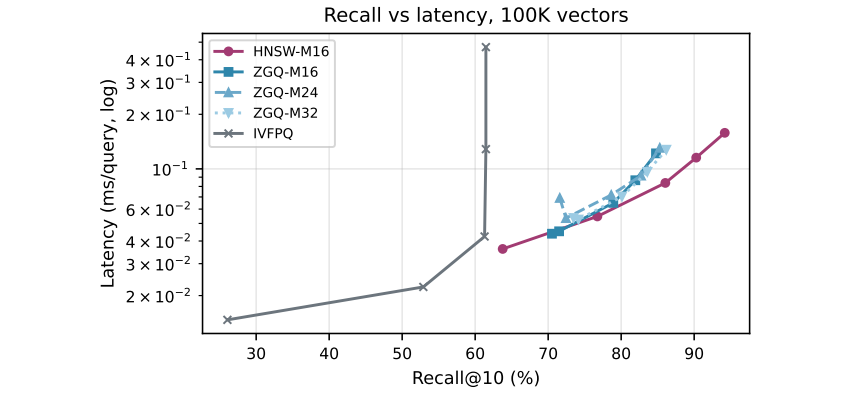


Figure 3: Recall vs latency (log scale) at 100K vectors, including IVF-PQ. No ZGQ setting dominates HNSW-M16; IVF-PQ is fast but saturates at 61.5% recall.

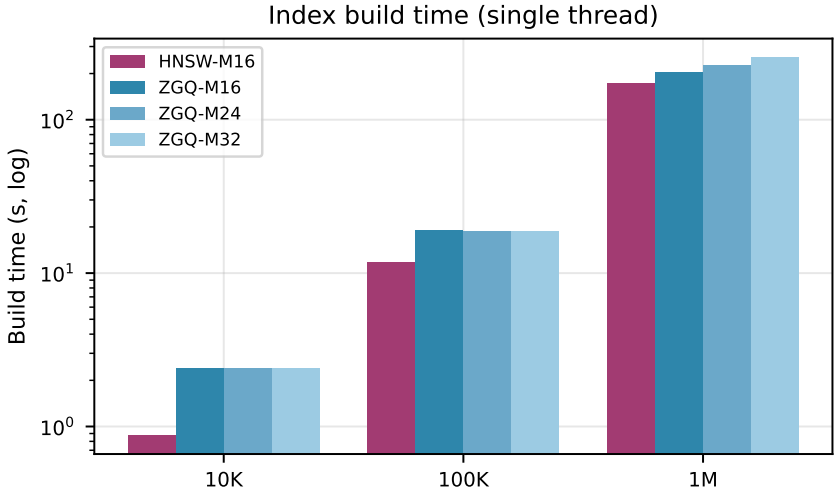


Figure 4: Index build time (log scale) per scale. ZGQ builds 1.2x to 2.7x slower than HNSW-M16. HNSW is C++ (hnswlib); ZGQ is Python with numba, so part of the gap is implementation overhead.

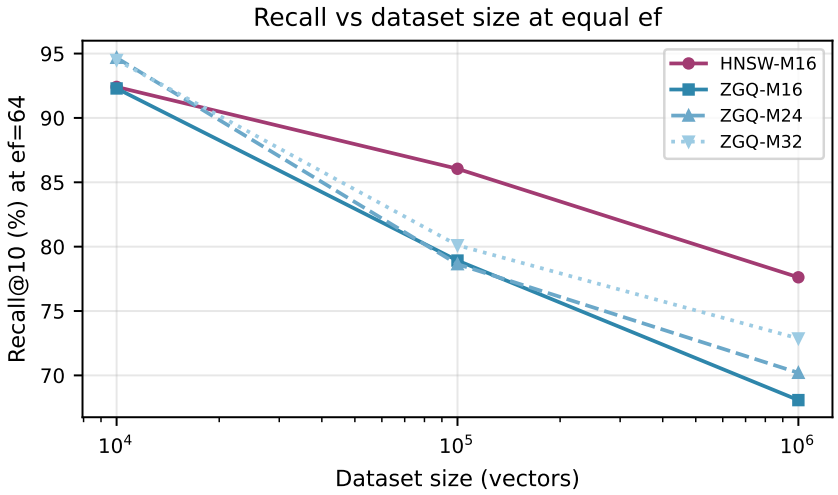


Figure 5: Recall@10 at ef = 64 vs dataset size. Recall falls for every graph index as the dataset grows, and ZGQ falls faster than HNSW-M16 in these runs. This does not support the claim that ZGQ's advantage grows with dataset size.

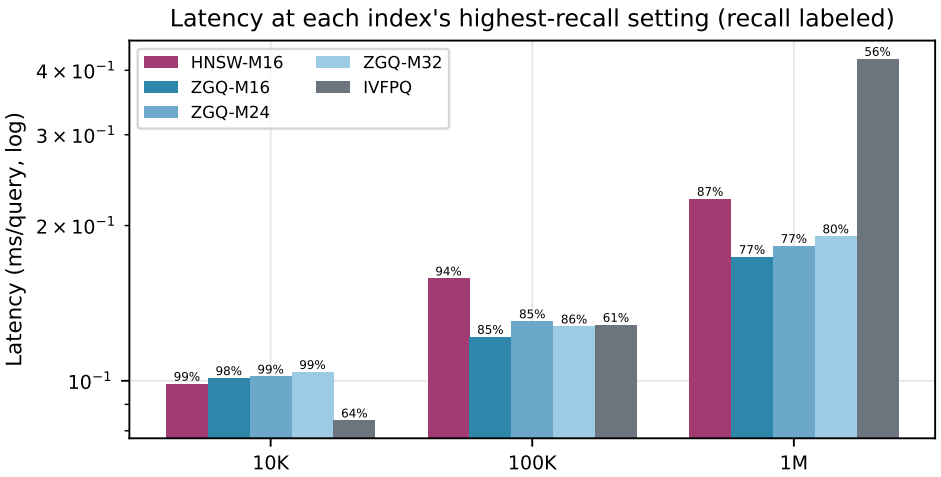


Figure 6: Latency at each index's highest-recall setting, with the recall labelled on each bar. IVF-PQ latency is low but its highest recall is 56 to 64%, so the bars are not at equal recall.

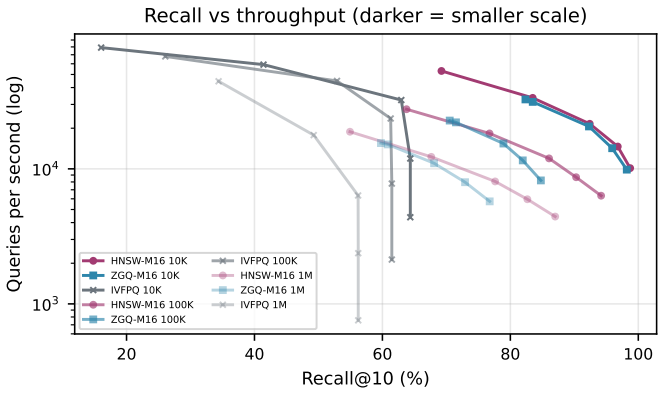


Figure 7: Recall vs throughput for HNSW-M16, ZGQ-M16 and IVF-PQ at all scales (darker is smaller). No memory axis is shown because RSS deltas on macOS are unreliable.

3. **Iso-Recall Efficiency** (Theorem 3.2): predicted a throughput gain at equal recall. The canonical run does not show it (iso-recall QPS ratio about 0.66 at 100K).

8.2 Key Findings

Measured result (2026-11-08)

Metric	ZGQ-M16 vs HNSW-M16
Recall@10 at equal ef	Equal at 10K; 7 to 10 points lower at 100K and 1M
Latency at equal ef	About 5% slower at 10K; 20 to 30% faster at 100K and 1M, at lower recall
Latency at equal recall	HNSW is at least as fast
Build time	ZGQ 1.2x to 2.7x slower
Memory	ZGQ uses far more (ordering only)

8.3 Practical Implications

- On this data ZGQ does not beat hnswlib HNSW on the recall and latency frontier at any tested scale.
- ZGQ is implemented in Python; part of the build-time and latency gap is implementation overhead, and a C++ port would be needed for a fair algorithmic comparison.
- The results are for synthetic data, one machine and 1000 queries. See the limitations in `docs/BENCHMARKS.md`.

A Derivation Details

A.1 Zone Density Factor α

The intra-zone density factor α is computed as:

$$\alpha = \frac{1}{Z} \sum_{z=1}^{Z} \frac{|\{(i,j) : \mathcal{Z}(x_i) = \mathcal{Z}(x_j) = z, (i,j) \in E\}|}{M \cdot n_z}$$

Assumed value: $\alpha \in [0.6, 0.7]$ for clustered data. This was not measured in the canonical run.

A.2 Zone-Guidance Efficiency Factor β

The zone-guidance efficiency is:

$$\beta = 1 - \frac{\mathbb{E}[\text{nodes visited in target zone}]}{\mathbb{E}[\text{total nodes visited}]}$$

Assumed value for $Z = 10$ zones with balanced sizes: $\beta \approx 0.3$. Not measured.

A.3 QPS Calculation

Queries per second:

$$\text{QPS} = \frac{n_{\text{queries}}}{T_{\text{total}}} = \frac{1000}{T_{\text{batch}}}$$

where T_{batch} is the median over 5 timed repeats after one warm-up pass.

B Reproducibility

```
cd v8
python -m benchmarks.run_benchmarks --scale 10k 100k 1m --out-dir results/2026-11-08
python -m visualization.generate_ieee_figures
python -m visualization.make_tables
# Figures are written to v8/figures/, tables to v8/tables/.
```